

Rendering Features

이 문서는 프로젝트의 `MeshDecal`, `ProjectionDecal`, `Fog`, `FXAA`, `Fireball` 기능을 코드 기준으로 설명한다.

여기서 `ProjectionDecal` 은 프로젝트 내부 명칭상 `DecalActor` / `UDecalComponent` 기반 데칼을 의미한다.

1. MeshDecal

개요

`MeshDecal` 은 데칼 볼륨 안에 들어온 실제 메시 삼각형을 잘라 별도의 데칼 메시를 생성한 뒤, 그 메시를 직접 렌더링하는 방식이다.

스크린 공간에 투영하는 방식이 아니라, 대상 메시 표면을 따라붙는 전용 지오메트리를 CPU에서 만들어 렌더링한다.

주요 클래스

- `AMeshDecalActor`
- `UMeshDecalComponent`
- `FDecalMeshSceneProxy`
- `Shaders/MeshDecalShader.hlsli`

동작 방식

1. `AMeshDecalActor` 가 생성되면 루트로 `UMeshDecalComponent` 를 가진다.
2. `UMeshDecalComponent::BeginPlay()` 또는 transform 변경 시 `UpdateDecalMeshData()` 가 호출된다.
3. 컴포넌트는 자신의 OBB와 겹칠 가능성이 있는 프리미티브를 월드에서 조회한다.
4. 후보 중 `UStaticMeshComponent` 만 대상으로 삼아 각 삼각형을 데칼 로컬 공간으로 변환한다.
5. 데칼 박스 내부에 들어오는 부분만 Sutherland-Hodgman 형태의 축별 클리핑으로 잘라낸다.
6. 잘린 다각형을 triangle fan으로 인덱싱해 별도 메시 버퍼를 생성한다.
7. 픽셀 셰이더는 데칼 텍스처를 샘플링하고, `Opacity` 와 `Fade` 상태를 반영해 알파 블렌딩한다.

특징

- 대상 표면에 밀착되는 실제 기하를 생성하므로 표면 추종성이 좋다.
- 정점 위치를 노멀 방향으로 약간 띄워 z-fighting을 줄인다.
- UV는 데칼 로컬 공간의 yz 평면 기준으로 계산된다.
- 에디터에서는 박스 형태의 데칼 범위를 디버그 라인으로 확인할 수 있다.

편집 가능 속성

- Texture
- Fade
- Opacity
- Opacity Rate

주의 사항

- 대상은 현재 `UStaticMeshComponent` 위주로 수집된다.
- transform이 바뀌면 데칼 메시를 다시 계산하므로 비용이 증가할 수 있다.
- `TickComponent()` 에서 `Opacity` 가 시간에 따라 감소하므로, 설정에 따라 자연 감쇠형 효과로 동작한다.

2. ProjectionDecal

개요

`ProjectionDecal` 은 큐브 볼륨을 화면에 렌더링한 뒤, 깊이 버퍼를 이용해 픽셀의 월드 좌표를 복원하고 이를 데칼 로컬 공간으로 변환해서 텍스처를 투영하는 방식이다.

프로젝트 코드에서는 `ADecalActor` 와 `UDecalComponent` 가 이 기능을 담당한다.

주요 클래스

- `ADecalActor`
- `UDecalComponent`
- `FDecalSceneProxy`
- `Shaders/ShaderDecal.hlsl`

동작 방식

1. `UDecalComponent` 는 큐브 메시를 데칼 볼륨으로 사용한다.
2. `FDecalSceneProxy` 는 뷰마다 `WorldToDecal` 행렬과 `fade` 상수를 `constant buffer`에 넣는다.
3. 데칼 패스에서는 `depth-stencil view`를 잠시 해제하고 `depth SRV`를 픽셀 셰이더에 바인딩한다.
4. 픽셀 셰이더는 화면 UV와 `scene depth`로 월드 좌표를 복원한다.
5. 복원한 월드 좌표를 `WorldToDecal` 로 변환해 데칼 로컬 좌표를 얻는다.
6. 로컬 좌표가 `[-0.5, 0.5]` 범위를 벗어나면 `clip`한다.
7. 범위 안이면 `YZ` 를 UV로 사용해 데칼 텍스처를 샘플링하고, 필요 시 `fade`를 적용한다.

특징

- 별도 데칼 메시를 만들지 않고 화면 공간에서 투영하므로 구현이 간결하다.

- 장면 깊이 기반이라 볼륨 내부의 표면에만 색이 입혀진다.
- 렌더 상태는 `FrontCull + AlphaBlendKeepAlpha` 를 사용한다.
- `FadeInner` , `FadeOuter` 를 이용한 스포트라이트형 가장자리 감쇠가 가능하다.

편집 가능 속성

- `Texture`
- `Fade`
- `Fade Inner`
- `Fade Outer`

MeshDecal과의 차이

- `ProjectionDecal` 은 depth 복원 기반의 투영 방식이다.
- `MeshDecal` 은 실제 잘린 데칼 메시를 생성해 렌더링한다.
- `ProjectionDecal` 은 구현이 가볍지만 깊이 기반 한계가 있고, `MeshDecal` 은 표면 밀착성이 좋지만 CPU 메시 생성 비용이 있다.

3. Fog

개요

Fog는 `Exponential Height Fog` 형태로 구현되어 있으며, 장면 depth를 기반으로 각 픽셀의 월드 위치를 복원한 뒤 높이에 따른 안개량을 계산하는 `fullscreen post-process` 패스이다.

주요 클래스

- `AExponentialHeightFogActor`
- `UHeightFogComponent`
- `FRenderCollector::CollectFog`
- `FRenderer::DrawHeightFog`
- `Shaders/HeightFog.hlsl`

동작 방식

1. `UHeightFogComponent` 가 렌더 상태 생성 시 `FScene` 에 등록된다.
2. 렌더 수집 단계에서 `FRenderCollector::CollectFog()` 가 현재 fog 파라미터를 `RenderBus` 에 기록한다.
3. Fog 패스에서 depth SRV를 바인딩하고 `fullscreen triangle`을 그린다.
4. 셰이더는 depth로부터 월드 좌표를 복원한다.
5. 카메라 높이, fog 높이, `FogDensity` , `FogHeightFalloff` 를 이용해 안개 적분값을 구한다.
6. 계산된 `opacity`를 `FogInscatteringColor` 와 함께 출력해 현재 컬러 버퍼에 블렌딩한다.

편집 가능 속성

- FogDensity
- FogHeightFalloff
- StartDistance
- FogCutoffDistance
- FogMaxOpacity
- FogInscatteringColor

특징

- exp2 기반 exponential height fog를 사용한다.
- sky 픽셀(depth=1)은 안개 색으로 채운다.
- Fog show flag가 꺼져 있으면 수집과 렌더링 모두 생략된다.
- 블렌드 상태는 알파를 보존하는 AlphaBlendKeepAlpha 를 사용한다.

현재 구현 특성

- 씬에서 FScene::GetFog() 가 첫 번째 fog 컴포넌트만 반환하므로 사실상 단일 fog 기준으로 동작한다.
- 안개 높이는 fog 액터의 월드 z 위치를 기준으로 계산된다.

4. FXAA

개요

FXAA는 장면 렌더링이 끝난 뒤 최종 컬러 텍스처에 대해 경계 기반 smoothing을 수행하는 post-process anti-aliasing이다.

주요 클래스

- FRenderer::DrawFXAA
- Shaders/FXAA.hlsl
- FRenderBus::SetFXAAEnabled
- 에디터 UI의 Enable FXAA

동작 방식

1. 렌더러는 현재 post-process 입력 SRV를 가져온다.
2. FXAA 패스에서 별도의 post-process RTV로 fullscreen triangle을 그린다.
3. 픽셀 셰이더는 주변 5개 샘플의 luminance를 비교해 edge 여부를 판정한다.
4. edge가 약하면 원본 색을 그대로 출력한다.

5. edge가 강하면 방향 벡터를 구해 주변을 추가 샘플링하고 보간된 결과를 출력한다.

특징

- 구현은 Simplified FXAA 3.11 계열이다.
- geometry 재렌더링 없이 후처리만으로 aliasing을 줄인다.
- 렌더 옵션의 bFXAA 가 꺼져 있으면 패스 자체를 건너뛴다.

장단점

- 장점: 비용이 비교적 낮고 적용이 간단하다.
- 단점: 텍스처 디테일이나 UI 가장자리까지 함께 부드러워질 수 있다.

5. Fireball

개요

Fireball은 장면 안에 배치된 구체형 액터를 단순히 그리는 기능이 아니라, 별도의 fullscreen 패스에서 주변 픽셀에 발광(light add) 효과를 더하는 후처리형 이펙트이다.

주요 클래스

- AFireBallActor
- UFireBallComponent
- FRenderCollector::CollectFireBall
- FRenderer::ExecuteFireBallPass
- Shaders/ShaderFireBall.hlsl

동작 방식

1. AFireBallActor 는 기본적으로 sphere static mesh와 UFireBallComponent 를 가진다.
2. 수집 단계에서 모든 UFireBallComponent 를 순회하며 중심 위치와 반경, 감쇠, 색, 세기를 FFireBallEntry 로 모은다.
3. Fireball 패스는 depth SRV와 현재 color SRV를 동시에 읽는다.
4. 셰이더는 각 픽셀의 월드 좌표를 복원한다.
5. 최대 16개의 fireball에 대해 픽셀과 중심 간 거리를 계산하고, 반경 안에서 pow 기반 falloff를 적용한다.
6. 누적된 발광량을 기존 base color에 더해 출력한다.

편집 가능 속성

- Intensity

- Radius
- Radius FallOff
- Color

특징

- 실질적으로는 additive glow/light injection 성격의 post-process 효과다.
- 여러 fireball의 영향을 한 패스에서 누적할 수 있다.
- 현재 constant buffer 기준 최대 16개까지 지원한다.

현재 구현 특성

- RadiusOfSource 값은 컴포넌트에는 있지만 현재 셰이더 계산에는 사용되지 않는다.
- Fireball 패스는 깊이 테스트 없이 후처리로 실행된다.
- scene depth 모드에서도 Fireball 패스는 유지되도록 렌더 루프에 포함되어 있다.

6. 에디터에서의 사용

배치

Jungle Control Panel 의 Spawn 메뉴에서 다음 액터를 직접 생성할 수 있다.

- Decal
- ExponentialHeightFog
- MeshDecal
- FireBall

토글

- Enable FXAA : 모든 레벨 뷰포트의 FXAA 사용 여부를 전환한다.
- Fog : 뷰포트 show flag에서 안개 표시를 켜고 끌 수 있다.

7. 요약

- MeshDecal : CPU에서 잘린 데칼 메시를 만들어 표면에 밀착시켜 렌더링
- ProjectionDecal : depth 복원 기반의 스크린/볼륨 투영 데칼
- Fog : exponential height fog post-process
- FXAA : 최종 컬러에 적용되는 경량 안티앨리어싱
- Fireball : 깊이 기반 월드 위치 복원 후 발광을 누적하는 후처리 효과